Conteneurs avancés et orchestration Kubernetes

Informatique — Licence 3 (L3)

Cours INF-L3-22

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

À propos de ce cours
Exploiter les conteneurs à grande échelle : rappels Docker (images, conteneurs, registres), pods et services Kubernetes, déploiements sans interruption, configuration et secrets, supervision d'un cluster — et les limites de l'orchestration face aux alternatives plus légères.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — Rappels Docker : images, conteneurs et registres
• Leçon 2 — Orchestration avec Kubernetes : pods et services
• Leçon 3 — Déploiements et mises à jour sans interruption
• Leçon 4 — Configuration et secrets dans les environnements conteneurisés
• Leçon 5 — Supervision et journalisation d'un cluster
• Leçon 6 — Limites de l'orchestration et alternatives plus légères
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

Introduction à la matière

Dans les cours précédents (INF-L2-20, INF-L3-05, INF-L3-12), tu as appris à empaqueter une application dans une image Docker et à la lancer dans un conteneur. Tant qu'il s'agit de quelques conteneurs sur une seule machine, des commandes manuelles suffisent. Mais imagine une application utilisée jour et nuit : dix conteneurs à surveiller, des mises à jour chaque semaine, une panne possible à tout moment. Qui relance un conteneur tombé à 3 heures du matin ? Qui répartit le trafic ? C'est exactement le travail d'un orchestrateur, et le plus répandu d'entre eux s'appelle Kubernetes.

Kubernetes ne lance pas seulement des conteneurs : il maintient en permanence un état désiré que tu déclares dans des fichiers YAML appelés manifestes. Tu écris « je veux trois exemplaires de mon application, joignables par ce nom stable », et le système compare sans relâche la réalité à ta déclaration pour corriger tout écart — un conteneur qui meurt est remplacé, un nœud qui tombe est contourné. Ce cours t'apprend à écrire ces manifestes correctement, à les faire évoluer sans interruption de service, et à observer ce qui se passe dans le cluster.

Reste lucide : Kubernetes est un outil puissant mais coûteux à apprendre et à exploiter. La dernière leçon est entièrement consacrée à ses limites et aux alternatives plus légères, car un bon ingénieur choisit l'outil proportionné au problème. Tout au long du cours, nous faisons évoluer « BiblioWeb », l'application de la bibliothèque de l'IAM déjà rencontrée dans les cours de cloud et de DevOps : d'abord une image Docker, puis un déploiement Kubernetes complet, supervisé et mis à jour sans coupure.

Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

- Distinguer image, conteneur et registre, et expliquer le rôle des étiquettes (tags) et des empreintes (digests).
- Expliquer le modèle déclaratif de Kubernetes : état désiré, contrôleurs et boucles de réconciliation.
- Écrire un Deployment et un Service concordants (labels, sélecteurs, ports) et diagnostiquer un Service sans points de terminaison.
- Configurer une mise à jour progressive (RollingUpdate) avec maxSurge et maxUnavailable, et revenir en arrière proprement.
- Séparer la configuration du code avec ConfigMap et Secret, en expliquant pourquoi un Secret n'est pas chiffré par défaut.
- Superviser un cluster : journaux, métriques, sondes de vie et de disponibilité, diagnostic avec kubectl.
- Évaluer quand Kubernetes est disproportionné et choisir une alternative adaptée (Docker Compose, conteneurs managés, k3s…).

Leçon 1 — Rappels Docker : images, conteneurs et registres

Une image Docker est le modèle figé d'une application : un empilement de couches en lecture seule (le système de fichiers de base, les bibliothèques, le code, la configuration par défaut) construit à partir d'un Dockerfile. Un conteneur est une instance vivante de cette image : on y ajoute une fine couche inscriptible et un processus isolé qui s'exécute. La même image peut donner naissance à dix conteneurs identiques ; détruire un conteneur ne modifie pas l'image. Cette distinction est la fondation de tout le cours : Kubernetes ne manipule jamais des conteneurs « artisanaux », il instancie des images.

Pour circuler entre machines, les images sont stockées dans des registres (registries) : Docker Hub en est le registre public le plus connu, mais les entreprises utilisent des registres privés (ceux des fournisseurs cloud, ou des outils comme Harbor) pour leurs images internes et pour des raisons de sécurité. Chaque image y est désignée par un nom et une étiquette (tag) : biblio/web:2.4. Le tag est une étiquette humaine, modifiable ; l'empreinte (digest, un hachage sha256 du contenu) identifie une image de façon unique et immuable. En production, on évite le tag « latest », qui ne dit rien sur le contenu réel : on déploie des versions explicites, traçables, que l'on peut re-déployer à l'identique.

Pourquoi ne pas s'arrêter là et lancer ses conteneurs à la main ? Parce qu'un conteneur seul ne sait ni se relancer après un crash, ni se multiplier quand le trafic augmente, ni recevoir une mise à jour sans coupure, ni être retrouvé par les autres services quand son adresse change. Sur une machine unique et pour quelques services, Docker Compose comble déjà une partie de ces manques. Mais dès que l'on vise la haute disponibilité sur plusieurs machines, il faut un chef d'orchestre : c'est le rôle de Kubernetes, qui prend en charge le placement, la surveillance, le redémarrage et la

mise en réseau des conteneurs.

L'essentiel
Image = modèle figé et versionné ; conteneur = instance en cours d'exécution ; registre = bibliothèque d'images partagée. Docker sait lancer des conteneurs, mais pas les orchestrer : ni auto-réparation, ni passage à l'échelle, ni mise à jour sans coupure.

Exemple — Construire, étiqueter, publier et lancer l'image de BiblioWeb

```
# 1. Construire l'image depuis le Dockerfile du projet
docker build -t biblio/web:2.4 .

# 2. La publier dans le registre prive de l'institut
docker tag biblio/web:2.4 registre.iam.ml/biblio/web:2.4
docker push registre.iam.ml/biblio/web:2.4

# 3. Verifier ce qui est reellement publie (le digest identifie
# l'image de facon unique, le tag n'est qu'une etiquette)
docker image inspect registre.iam.ml/biblio/web:2.4
# "RepoDigests": ["registre.iam.ml/biblio/web@sha256:9f2c...a41d"]

# 4. Lancer UN conteneur, a la main, pour tester
docker run -d --name biblio-test -p 8080:8080 registre.iam.ml/biblio/web:2.4
docker ps # le conteneur tourne
docker logs biblio-test
docker stop biblio-test && docker rm biblio-test

# Limite : si ce conteneur tombe a 3 h du matin, personne ne le
# relance. C'est ce probleme que Kubernetes resout (lecon 2).
```

À retenir

- Une image est immuable et en lecture seule ; un conteneur ajoute une couche inscriptible temporaire par-dessus.
- Un tag est une étiquette modifiable, un digest est une empreinte unique : en production on déploie des versions explicites, jamais « latest ».
- Docker Compose suffit pour quelques conteneurs sur une machine ; l'orchestration commence quand il faut réparer, répartir et mettre à jour automatiquement.

Exercice

Un collègue affirme : « J'ai corrigé le bug directement dans le conteneur en production, tout va bien. » Explique en quelques lignes pourquoi cette correction va disparaître, et décris la démarche correcte pour déployer un correctif durable avec Docker.

✔ Corrigé
La correction a été faite dans la couche inscriptible du conteneur, qui est temporaire : au prochain redémarrage, remplacement ou passage à l'échelle, un nouveau conteneur sera créé à partir de l'image d'origine — sans le correctif. La couche inscriptible ne fait pas partie de l'image et n'est jamais poussée dans le registre telle quelle. La démarche durable est : corriger le code source ou le Dockerfile, reconstruire une NOUVELLE image avec un nouveau tag (par exemple biblio/web:2.4.1), la pousser dans le registre, puis redéployer à partir de cette image. Règle d'or : on ne modifie jamais un conteneur en production, on remplace l'image.

Leçon 2 — Orchestration avec Kubernetes : pods et services

Un cluster Kubernetes est un ensemble de machines (les nœuds) pilotées par un plan de contrôle (control plane) qui expose une API. On ne donne pas d'ordres impératifs au cluster (« lance ceci, puis cela ») : on lui soumet des manifestes déclaratifs qui décrivent l'état désiré. Des contrôleurs comparent en permanence l'état réel à l'état désiré et agissent pour les faire coïncider : c'est la boucle de réconciliation. Si un pod meurt, le contrôleur le constate et en crée un autre ; si tu modifies le manifeste, il fait évoluer le système vers la nouvelle cible. Tu déclares le « quoi », Kubernetes se charge du « comment », en continu.

L'unité de base que Kubernetes déploie est le pod : le plus petit objet déployable, qui regroupe un ou plusieurs conteneurs partageant le même réseau (une adresse IP commune) et pouvant partager des volumes. En pratique, un pod contient le plus souvent un seul conteneur principal, éventuellement accompagné d'un conteneur auxiliaire très lié. Les pods sont éphémères : ils naissent, meurent et sont remplacés, avec une nouvelle adresse IP à chaque fois. C'est pourquoi on ne déploie presque jamais un pod « nu » : on passe par un Deployment, qui maintient le nombre voulu de pods identiques (les réplicas) selon un modèle (template), et les remplace au fur et à mesure.

Comment joindre des pods qui changent d'adresse sans cesse ? Par un Service : un nom DNS stable et une adresse IP virtuelle dans le cluster, qui répartit les connexions vers les pods désignés par un sélecteur de labels. Les labels sont des étiquettes clé/valeur posées sur les objets ; le sélecteur du Service choisit tous les pods portant ces étiquettes. Point critique : si le sélecteur ne correspond aux labels d'aucun pod, le Service existe mais ne mène nulle part — on dit qu'il n'a aucun point de terminaison (endpoint), et l'application devient injoignable alors que tout semble « créé » sans erreur. Le Service de base, ClusterIP, est interne au cluster ; NodePort et LoadBalancer servent à exposer vers l'extérieur.

L'essentiel
Kubernetes est déclaratif : tu décris l'état désiré, les contrôleurs réconcilient sans relâche. Le pod est la plus petite unité déployable, éphémère ; le Deployment maintient N réplicas d'un modèle de pod ; le Service donne un nom stable et choisit ses pods par un sélecteur de labels — sans correspondance, aucun trafic.

Exemple — Déployer BiblioWeb : un Deployment et le Service qui l'expose

```
apiVersion: apps/v1
kind: Deployment
metadata:
name: biblio-web
labels:
app: biblio-web
spec:
replicas: 3
selector:
matchLabels:
app: biblio-web
template:
metadata:
labels: # <- les labels poses sur chaque pod cree
app: biblio-web
couche: presentation
spec:
containers:
- name: web
image: registre.iam.ml/biblio/web:2.4
ports:
- containerPort: 8080
---
apiVersion: v1
kind: Service
metadata:
name: biblio-web
spec:
type: ClusterIP
selector:
app: biblio-web # <- doit correspondre aux labels des pods
ports:
- name: http
port: 80 # port du Service dans le cluster
targetPort: 8080 # port du conteneur dans le pod
```

À retenir

• Le pod est la plus petite unité déployable de Kubernetes ; il est éphémère et son adresse IP change à chaque remplacement.

• Le sélecteur d'un Service (ou d'un Deployment) doit correspondre EXACTEMENT aux labels du modèle de pod, sinon l'objet est vide de cibles.

• État désiré + boucles de réconciliation : après un incident, Kubernetes revient tout seul à la cible déclarée, sans intervention humaine.

Exercice

Après avoir appliqué ses deux manifestes, une étudiante constate que son Service « catalogue » affiche « Endpoints: <none> » alors que trois pods tournent et sont en état Running. Le Service porte le sélecteur `app: catalogue-api`, et les pods portent

les labels `app: catalogue` et `couche: api`. Explique la cause du problème et propose la correction la plus propre.

✔ Corrigé
Le Service ne trouve aucun pod car son sélecteur exige le label `app: catalogue-api`, qu'aucun pod ne porte : les pods ont `app: catalogue`. Le sélecteur et les labels doivent être identiques, caractère pour caractère ; Kubernetes ne signale pas d'erreur, il se contente de n'avoir aucune cible — d'où le piège. Correction propre : aligner les deux côtés sur la même convention. Le plus sûr est de corriger le sélecteur du Service en `app: catalogue` (les pods existent déjà avec ces labels), puis de vérifier avec `kubectl get endpoints catalogue` que les adresses des pods apparaissent, ou avec `kubectl describe service catalogue`. À l'avenir, on définit les labels une seule fois et on les copie sans les retaper, et on contrôle systématiquement les endpoints après chaque déploiement.

Leçon 3 — Déploiements et mises à jour sans interruption

Mettre à jour une application en production sans la rendre indisponible est l'un des apports majeurs de l'orchestration. Quand le modèle de pod change (nouvelle image, par exemple), le Deployment ne remplace pas tout d'un coup : il crée un nouveau ReplicaSet pour la nouvelle version et fait basculer progressivement les réplicas de l'ancien vers le nouveau, en gardant l'ancien prêt à reprendre du service tant que la transition n'est pas terminée. Cette stratégie par défaut s'appelle RollingUpdate — mise à jour roulante. L'ancienne stratégie Recreate, qui éteint tout avant de rallumer, provoque une coupure et ne se choisit que pour des applications très particulières.

Deux paramètres règlent la progressivité : maxSurge, le nombre de pods supplémentaires (au-delà du nombre voulu) que l'on autorise pendant la transition, et maxUnavailable, le nombre de pods que l'on accepte de voir indisponibles simultanément. Avec replicas: 3, maxSurge: 1 et maxUnavailable: 0, la mise à jour crée d'abord un quatrième pod en nouvelle version, n'éteint un ancien que lorsqu'un nouveau est prêt, et garantit qu'au moins trois pods servent le trafic à tout instant. Ces paramètres s'expriment en nombres absolus ou en pourcentages du nombre de réplicas : maxSurge est arrondi au supérieur, maxUnavailable à l'inférieur, pour ne jamais dépasser l'indisponibilité tolérée.

Encore faut-il savoir quand un nouveau pod est réellement prêt à recevoir des utilisateurs : démarré ne veut pas dire prêt (l'application charge peut-être encore ses données). La sonde de disponibilité (readiness probe) interroge le pod — par exemple une requête HTTP sur un chemin /sante — et le Service ne lui envoie du trafic que lorsqu'elle répond positivement. Pendant une mise à jour roulante, c'est elle qui déclenche l'extinction progressive des anciens pods. Et si la nouvelle version se révèle défaillante, l'historique des révisions permet un retour en arrière immédiat (kubectl rollout undo) vers la version précédente, sans reconstruire quoi que ce soit.

L'essentiel
Une mise à jour roulante remplace les pods un par un : maxSurge autorise des pods en plus, maxUnavailable borne les pods indisponibles. La readiness probe décide quand un nouveau pod peut recevoir du trafic, et l'historique des révisions rend le retour en arrière quasi instantané.

Exemple — Stratégie RollingUpdate de BiblioWeb et bascule de la version 2.4 vers 2.5

```
# Extrait complete du Deployment de la lecon 2 :
spec:
replicas: 3
strategy:
type: RollingUpdate
rollingUpdate:
maxSurge: 1 # au plus 4 pods pendant la transition
maxUnavailable: 0 # jamais moins de 3 pods prets
template:
spec:
containers:
- name: web
image: registre.iam.ml/biblio/web:2.5 # <- seule ligne changee
readinessProbe:
httpGet: { path: /sante, port: 8080 }
initialDelaySeconds: 5
periodSeconds: 5

# Deroulement observe (les anciens pods ne s'eteignent
# qu'au fur et a mesure que les nouveaux deviennent prets) :
# t1 : nouveau ReplicaSet 0 -> 1 pod (total 4, surge respecte)
# t2 : le pod 2.5 est Ready -> ancien ReplicaSet 3 -> 2 pods
# t3 : nouveau 1 -> 2 pods ; ancien 2 -> 1 pod
# t4 : nouveau 2 -> 3 pods ; ancien 1 -> 0 pod : bascule terminee
#
# Suivi et, si la 2.5 se comporte mal, retour arriere :
# kubectl rollout status deployment/biblio-web
# kubectl rollout undo deployment/biblio-web
```

À retenir

- maxSurge = pods supplémentaires autorisés pendant la mise à jour ; maxUnavailable = pods indisponibles tolérés en même temps.
- Sans readiness probe, Kubernetes envoie du trafic à des pods démarrés mais pas encore prêts : erreurs garanties pendant les déploiements.
- Chaque modification du modèle de pod crée une révision : kubectl rollout undo revient à la précédente en quelques secondes.

Exercice

Un Deployment compte 4 réplicas, avec maxSurge: 25% et maxUnavailable: 25%. Pendant une mise à jour roulante : 1) quel est le nombre maximal de pods existant simultanément ? 2) quel est le nombre minimal de pods disponibles garanti ? 3)

pourquoi Kubernetes arrondit-il maxSurge au supérieur et maxUnavailable à l'inférieur, et pas l'inverse ?

✔ Corrigé
25 % de 4 réplicas = 1 pod dans les deux cas. 1) Maximum de pods pendant la transition : 4 + 1 = 5. 2) Minimum de pods disponibles : 4 − 1 = 3. 3) Les arrondis protègent la promesse faite par l'exploitant : arrondir maxUnavailable au supérieur reviendrait à tolérer plus d'indisponibilité que demandé (avec 6 réplicas et 25 %, on obtiendrait 2 pods indisponibles au lieu d'1,5 → Kubernetes retient 1), tandis qu'arrondir maxSurge à l'inférieur pourrait bloquer la progression (avec 3 réplicas et 25 %, 0,75 arrondi à 0 empêcherait de créer le premier nouveau pod si maxUnavailable vaut 0). La règle réelle : maxSurge est arrondi au supérieur, maxUnavailable à l'inférieur.

Leçon 4 — Configuration et secrets dans les environnements conteneurisés

Une bonne image est générique : la même image de BiblioWeb doit servir en test et en production, seuls changent la langue affichée, le nombre de résultats par page ou l'adresse de la base de données. Faire varier ces réglages sans reconstruire l'image, c'est le rôle de la configuration externalisée. Kubernetes propose l'objet ConfigMap pour les données de configuration non sensibles : paires clé/valeur injectées dans les pods comme variables d'environnement ou comme fichiers montés. Modifier une ConfigMap ne reconstruit rien ; selon le mode d'injection, le pod lit la nouvelle valeur au redémarrage (variables d'environnement) ou dans le fichier mis à jour (montage en volume).

Les données sensibles — mots de passe, jetons d'API, clés privées — relèvent d'un autre objet : le Secret. Attention, et c'est un point que ce cours souligne fortement : par défaut, un Secret Kubernetes n'est PAS chiffré. Ses valeurs sont simplement encodées en base64 dans le manifeste, un encodage lisible par n'importe qui en une commande. Un Secret YAML commité dans un dépôt Git est donc un mot de passe en clair, point final. Kubernetes offre des garde-fous qu'il faut activer et comprendre : chiffrement au repos dans etcd, règles d'accès RBAC limitant qui peut lire les Secrets, et en production sérieuse, gestionnaires de secrets externes qui injectent les valeurs sans les stocker dans le cluster.

Configurer proprement, c'est aussi savoir ce qui ne va ni dans une ConfigMap ni dans un Secret : les secrets ne doivent jamais passer dans l'image (un docker history suffit à les retrouver), ni dans les journaux, ni dans le code source. La pratique saine tient en trois gestes : les valeurs non sensibles en ConfigMap, les valeurs sensibles en Secret consommé par variable d'environnement ou fichier monté, et aucune des deux dans le dépôt de code — on y dépose seulement un fichier d'exemple avec des valeurs fictives. En cas de fuite suspectée, on fait tourner (rotation) le secret : créer de nouvelles valeurs, redéployer, révoquer les anciennes.

L'essentiel
ConfigMap = configuration non sensible, externalisée de l'image. Secret = données sensibles, mais seulement encodées en base64 par défaut, PAS chiffrées : il faut le chiffrement au repos dans etcd, le RBAC et, en production, un gestionnaire externe. Ni secret dans l'image, ni dans Git, ni dans les logs.

Exemple — La configuration de BiblioWeb : une ConfigMap, un Secret, et leur consommation

```
apiVersion: v1
kind: ConfigMap
metadata:
name: biblio-config
data:
LANGUE_DEFAUT: "fr"
LIVRES_PAR_PAGE: "20"
MESSAGE_ACCUEIL: "Bienvenue a la bibliotheque IAM"
---
apiVersion: v1
kind: Secret
metadata:
name: biblio-secrets
type: Opaque
data:
# base64 = ENCODAGE, pas un chiffrement : "biblio2026"
# est une valeur FICTIVE d'exemple, jamais la vraie.
mot-de-passe-bdd: YmlibGlvMjAyNg==
stringData:
jeton-api: "valeur-fictive-a-remplacer"

# Consommation dans le modele de pod du Deployment :
# envFrom:
# - configMapRef: { name: biblio-config }
# env:
# - name: MOT_DE_PASSE_BDD
# valueFrom:
# secretKeyRef: { name: biblio-secrets, key: mot-de-passe-bdd }
```

À retenir

- La même image sert partout ; seuls ConfigMap et Secrets changent selon l'environnement.
- base64 n'est pas un chiffrement : kubectl get secret -o yaml révèle les valeurs à quiconque a les droits de lecture — RBAC et chiffrement au repos sont indispensables.
- Un secret qui a fuité ne se « cache » pas, il se remplace : rotation = nouvelles valeurs, redéploiement, révocation des anciennes.

Exercice

Pour faire tourner BiblioWeb chez un hébergeur, un étudiant propose de : (a) mettre l'URL publique du site dans le Secret ; (b) écrire le mot de passe de la base, encodé en

base64, dans le fichier secret.yaml commité sur le GitHub public du projet « pour ne pas le perdre » ; (c) passer la langue d'affichage par une variable d'environnement codée en dur dans l'image. Corrige chacune des trois propositions.

✔ Corrigé
(a) L'URL publique n'a rien de sensible : elle va dans la ConfigMap, pas dans le Secret — réserver les Secrets aux vraies données confidentielles limite leur surface d'exposition. (b) C'est la faute la plus grave : base64 se décode en une commande (echo YmlibGlvMjAyNg== | base64 -d), donc committer ce fichier sur un dépôt public revient à publier le mot de passe ; de plus, l'historique Git conserve le fichier même après suppression. Il faut : sortir le fichier du dépôt (et le déclarer dans .gitignore), considérer le mot de passe comme compromis et le changer (rotation), puis fournir le Secret par un canal sûr (commande kubectl create secret hors Git, ou gestionnaire de secrets). (c) Une valeur codée en dur dans l'image oblige à reconstruire et republier l'image pour chaque changement de langue : on la déplace dans la ConfigMap, injectée par envFrom, et la même image sert alors à tous les environnements.

Leçon 5 — Supervision et journalisation d'un cluster

Exploiter un cluster sans le voir est impossible : la supervision répond à trois questions — le système est-il en bonne santé (métriques), que s'est-il passé (journaux), et où le temps est-il passé dans une requête (traces) ? Ce cours se concentre sur les deux premiers piliers. Les métriques sont des mesures chiffrées régulières (processeur, mémoire, nombre de requêtes, taux d'erreurs). L'écosystème Kubernetes s'appuie le plus souvent sur Prometheus, qui interroge périodiquement les composants et les applications, et sur Grafana pour les tableaux de bord ; le serveur de métriques intégré (metrics-server) alimente quant à lui les commandes kubectl top et l'autoscaling.

Côté journaux, la règle d'or des conteneurs est d'écrire sur la sortie standard (stdout) et l'erreur standard (stderr), jamais dans un fichier interne au conteneur : un conteneur détruit emporte ses fichiers, alors que ses flux sont capturés par la plateforme. kubectl logs lit le journal du conteneur actuel — et avec l'option --previous, celui du conteneur précédent, précieux quand un pod redémarre en boucle. À l'échelle d'un cluster, un collecteur (Fluent Bit, par exemple, installé sur chaque nœud) expédie tous les journaux vers un moteur central (Elasticsearch, Loki) où l'on recherche par service, par niveau ou par identifiant de requête corrélant les étapes d'une même demande.

La première ligne de défense reste les sondes : la readiness probe (vue à la leçon 3) retire un pod malade du trafic, et la liveness probe demande à Kubernetes de redémarrer un conteneur bloqué — utile contre les interblocages, dangereuse si elle est trop agressive, car elle peut transformer un simple ralentissement en boucle de redémarrages. Le diagnostic d'un pod anormal suit un rituel : kubectl get pods pour l'état global (Running, Pending, CrashLoopBackOff, ImagePullBackOff sont des diagnostics en soi), kubectl describe pod pour les événements (échec de tirage d'image, mémoire insuffisante, sonde en échec), puis kubectl logs. Savoir lire ces trois sources avant de « réparer » au hasard est la compétence d'exploitation n° 1.

L'essentiel
Superviser = métriques (Prometheus/Grafana, kubectl top), journaux (stdout/stderr, kubectl logs, collecte centralisée) et sondes (readiness pour le trafic, liveness pour redémarrer). Face à un pod malade : get pour l'état, describe pour les événements, logs pour le récit — dans cet ordre.

Exemple — Diagnostiquer BiblioWeb avec kubectl : état, ressources et journaux structurés

```
$ kubectl get pods -l app=biblio-web
NAME READY STATUS RESTARTS AGE
biblio-web-7d9c4f6b5-x2k1a 1/1 Running 0 12m
biblio-web-7d9c4f6b5-m8qzp 1/1 Running 2 12m
biblio-web-7d9c4f6b5-t4wne 0/1 CrashLoopBackOff 5 12m

# Le 3e pod redemarre en boucle : on lit le journal du
# conteneur PRECEDENT, celui qui vient de mourir
$ kubectl logs biblio-web-7d9c4f6b5-t4wne --previous
{"ts":"2026-10-11T08:15:22Z","niveau":"ERREUR","service":"biblio-web",
"msg":"connexion a la base impossible","hote":"biblio-bdd","duree_ms":3001}

$ kubectl describe pod biblio-web-7d9c4f6b5-t4wne
# Warning BackOff ... restarting failed container
# -> la cause est cote base de donnees, pas dans le pod :
# on verifie le Service biblio-bdd et ses endpoints.

$ kubectl top pods -l app=biblio-web
# consommation CPU/memoire reelle, comparee aux requests/limits
```

À retenir

- Les journaux vont sur stdout/stderr : tout fichier écrit dans le conteneur disparaît avec lui.
- CrashLoopBackOff = le conteneur démarre puis meurt en boucle ; kubectl logs --previous lit le journal du conteneur mort, describe montre les événements.
- Une liveness probe trop agressive transforme un ralentissement en boucle de redémarrages : elle se règle avec autant de soin que le code.

Exercice

Après une mise à jour, tous les pods de BiblioWeb passent en ImagePullBackOff et l'ancienne version continue seule à servir le trafic. 1) Que signifie cet état ? 2) Quelles sont les deux causes les plus fréquentes ? 3) Quelle commande permet de confirmer la cause exacte avant de corriger ?

✔ Corrigé
1) ImagePullBackOff signifie que Kubernetes n'a pas réussi à télécharger l'image du conteneur et réessaie avec des délais croissants ; le pod ne démarre jamais. L'ancienne version continue de servir parce que la mise à jour roulante n'éteint les anciens pods qu'au fur et à mesure que les nouveaux deviennent prêts — c'est exactement la protection offerte par maxUnavailable. 2) Causes fréquentes : un nom d'image ou un tag inexistant (faute de frappe, version jamais poussée dans le registre), ou un registre privé dont les identifiants de tirage (imagePullSecrets) sont absents ou expirés. 3) kubectl describe pod <nom> : la section Events affiche le message d'erreur du tirage (manifest unknown, unauthorized…), ce qui tranche entre les deux causes avant toute correction — on corrige ensuite le tag ou le secret de tirage, jamais au hasard.

Leçon 6 — Limites de l'orchestration et alternatives plus légères

Kubernetes rend de grands services, mais il a un prix, et il faut le regarder en face. Prix d'apprentissage : pods, services, déploiements, ingress, stockage persistant, RBAC — la surface de concepts est vaste et chaque couche peut tomber en panne. Prix d'exploitation : faire tourner le plan de contrôle, le mettre à jour, le sécuriser et le superviser demande du temps d'équipe qualifiée, qu'un cluster soit utilisé par trois ou par trois cents applications. Prix en ressources enfin : l'orchestrateur et ses composants consomment une part non négligeable des machines. Pour une petite application, ces coûts fixes peuvent dépasser le bénéfice — un peu comme embaucher un régiment de logisticiens pour un seul camion de livraison.

Les alternatives existent à tous les étages. Sur une seule machine, Docker Compose orchestre honnêtement quelques conteneurs (définis dans un fichier YAML, relancés ensemble) : c'est l'outil de la grande majorité des petites applications, des sites d'associations et des environnements de développement. Les services de conteneurs managés des fournisseurs cloud (du type « exécute mon image, je paie à l'usage ») offrent le passage à l'échelle et la haute disponibilité sans administrer de cluster. Entre les deux, des orchestrateurs plus sobres — Docker Swarm, Nomad — ou un Kubernetes allégé comme k3s couvrent les besoins intermédiaires avec une courbe d'apprentissage plus douce.

Choisir devient alors un exercice d'ingénierie, pas de mode. Les critères qui comptent : le nombre d'applications et de services à faire cohabiter, l'exigence réelle de disponibilité (un site vitrine et un paiement en ligne ne vivent pas sous la même contrainte), la taille et l'expérience de l'équipe qui exploitera la plateforme à 3 heures du matin, et la croissance prévisible. Kubernetes s'impose quand de nombreuses équipes déploient de nombreux services, avec de fortes exigences de résilience et de montée en charge. Dans les autres cas, la solution la plus simple qui répond au besoin est la meilleure : on peut toujours migrer plus tard, et une équipe qui maîtrise Compose aujourd'hui vaut mieux qu'un cluster Kubernetes abandonné faute de compétences.

L'essentiel
Kubernetes coûte : apprentissage, exploitation, ressources. Compose suffit à la plupart des petites applications ; les conteneurs managés offrent l'échelle sans le cluster ; k3s, Swarm ou Nomad occupent le milieu. On choisit selon le besoin réel, pas selon la mode — et l'on peut migrer quand le besoin grandit.

Exemple — Choisir l'outil selon le contexte : quatre situations tranchées

```
SITUATION CHOIX CONSEILLE
------------------------------------------------ --------------------
Site vitrine d'une PME : 1 application web 1 machine +
+ 1 base de donnees, quelques visites/jour Docker Compose

Application metier d'une administration : Conteneurs manages
equipe de 2 developpeurs, disponibilite du cloud : l'equipe
bureautique, aucune envie d'administrer deploie, le fournisseur
un cluster exploite

Plateforme de paiement : 25 services, Kubernetes :
plusieurs equipes, trafic en forte croissance, auto-reparation, mises
astreinte 24/7 organisee a jour sans coupure,
autoscaling

Laboratoire d'une universite : petites k3s (Kubernetes leger)
applications reparties sur quelques machines ou Docker Swarm :
peu puissantes, etudiants administrateurs concepts reels,
exploitation sobre

Critere de decision, dans l'ordre :
equipe disponible pour exploiter > exigence de
disponibilite > nombre de services > croissance
```

À retenir

- Le coût d'un cluster ne se mesure pas en licences mais en compétences et en temps d'exploitation, nuit et week-end compris.
- Docker Compose n'est pas un « sous-Kubernetes » : c'est la bonne réponse, complète, pour une machine et quelques services.
- Migrer de Compose vers Kubernetes reste possible quand le besoin grandit ; l'inverse (un cluster surdimensionné abandonné) est le vrai échec.

Exercice

Une coopérative agricole veut publier les prix du marché : un site mis à jour chaque matin, consulté par quelques centaines de personnes, tenu par un informaticien à mi-temps. Un prestataire propose un cluster Kubernetes « aux standards de l'industrie ». Donne ton avis argumenté en quatre à cinq lignes, et propose la solution que tu retiendrais.

✔ Corrigé
L'avis est défavorable : les besoins (un site, un rythme quotidien, quelques centaines de visites, un exploitant à mi-temps) n'exigent ni auto-réparation multi-machines, ni mises à jour sans coupure, ni autoscaling ; en revanche le cluster exigerait des compétences et une astreinte que la coopérative n'a pas, avec un risque réel d'abandon ou de faille non corrigée. La solution proportionnée : une seule machine (ou un hébergement managé) avec Docker Compose pour le site et sa base, des sauvegardes automatiques et une supervision minimale. « Standard de l'industrie » n'est pas un argument : le bon standard est l'adéquation au besoin. Si le service grandit un jour (application mobile, paiements), on réévaluera — la conteneurisation faite dès maintenant rendra la migration possible.

Travaux dirigés (TD)

TD 1 — Le Service muet : diagnostiquer un déploiement qui ne reçoit aucun trafic

Sur le cluster de travaux pratiques, l'équipe déploie l'API « catalogue » de la bibliothèque. Les trois pods affichent Running et 1/1 Ready, mais le Service catalogue-api répond par des délais dépassés, et `kubectl get endpoints catalogue-api` affiche <none>. On te donne les extraits des manifestes : le Deployment crée des pods étiquetés `app: catalogue`, `couche: api`, écoutant sur le port 8080 ; le Service porte `selector: {app: catalogue, niveau: api}` et `ports: [{port: 80, targetPort: 8000}]`. Identifie TOUTES les fautes, explique pour chacune le symptôme qu'elle produit, puis écris les lignes corrigées du Service.

✔ Corrigé du TD 1
Il y a deux fautes indépendantes. Faute n° 1 : le sélecteur du Service exige `niveau: api`, label que les pods ne portent pas (ils portent `couche: api`) — le sélecteur doit être un sous-ensemble exact des labels des pods. Aucun pod n'étant sélectionné, le Service n'a aucun endpoint : c'est le symptôme observé (délais dépassés, endpoints <none>) malgré des pods en parfaite santé. Faute n° 2, encore invisible tant que la première n'est pas corrigée : targetPort vaut 8000 alors que l'application écoute sur 8080 ; même avec des endpoints, les connexions arriveraient sur un port fermé et seraient refusées. Service corrigé : `selector: {app: catalogue}` (on peut aussi ajouter `couche: api`, présent sur les pods) et `targetPort: 8080`. Vérification finale obligatoire : `kubectl get endpoints catalogue-api` doit lister les adresses des trois pods, puis un test réel (kubectl port-forward ou un pod de test avec curl) confirme le trafic. Leçon : un déploiement Kubernetes se valide par les endpoints et un appel réel, jamais par le seul « Running » des pods.

TD 2 — Dimensionner BiblioWeb pour la rentrée : réplicas, ressources et autoscaling

Pendant l'année, BiblioWeb sert 300 requêtes par minute avec 3 réplicas ; chaque pod traite confortablement 150 requêtes par minute tant que ses demandes de ressources (requests : 250m de CPU, 256 Mio) sont honorées. À la rentrée, on attend 1 200

requêtes par minute. 1) Combien de réplicas faut-il au minimum pour absorber ce trafic, en gardant une marge d'un pod « de secours » ? 2) Sur un nœud disposant de 4 CPU et 8 Gio de mémoire allouables, combien de ces pods peut-on placer au maximum, et quelle ressource sature en premier ? 3) Plutôt que de figer ce nombre, propose le réglage d'un autoscaler horizontal (HPA) cohérent avec cette étude.

✔ Corrigé du TD 2
1) Capacité nécessaire : 1 200 ÷ 150 = 8 pods pour le trafic ; avec un pod de marge (panne ou mise à jour sans dégrader le service) : 9 réplicas. 2) Placement par nœud : par le CPU, 4 000m ÷ 250m = 16 pods ; par la mémoire, 8 192 Mio ÷ 256 Mio = 32 pods. Le CPU sature en premier : 16 pods au maximum par nœud. Les 9 réplicas tiennent donc sur un seul nœud du point de vue des requests — mais la haute disponibilité impose de les répartir sur au moins deux nœuds, sinon la panne du nœud emporte tout le service. 3) HPA cohérent : minReplicas: 4 (le niveau creux de l'année, avec marge), maxReplicas: 12 (au-delà des 9 calculés, pour absorber un pic imprévu tout en bornant le coût), cible d'utilisation CPU à 60 % des requests : le HPA ajoute des pods dès que la charge moyenne dépasse durablement 60 % de 250m par pod, donc avant saturation réelle. On vérifie après une semaine de rentrée les métriques réelles (kubectl top, Prometheus) et l'on ajuste requests et seuils : le dimensionnement est une boucle d'observation, pas un calcul fait une fois pour toutes.

Travaux pratiques (TP)

TP 1 — Déployer BiblioWeb sur un cluster local et prouver sa résilience

Sur une machine de TP équipée de Docker, installe un cluster local à un nœud (Minikube ou kind, au choix). Travail demandé : 1) déploie BiblioWeb avec le Deployment, le Service, la ConfigMap et le Secret du cours (crée le Secret par kubectl create secret, sans le committer) ; 2) vérifie les endpoints du Service et appelle l'application (port-forward) ; 3) passe le déploiement à 5 réplicas et observe la création des pods ; 4) déclenche une mise à jour vers une image de version différente avec maxUnavailable: 0 et suis le rollout ; 5) supprime brutalement un pod (kubectl delete pod) et chronomètre sa recréation ; 6) simule une erreur : applique un Service dont le sélecteur est volontairement faux, constate les endpoints, puis répare. Livrable : un compte rendu avec les commandes, les sorties obtenues et une phrase d'explication par étape.

✔ Attendus du TP 1
Attendus : à l'étape 1, les quatre objets sont créés (kubectl get deploy,svc,cm,secret) et le Secret n'apparaît dans AUCUN fichier du dépôt Git du TP. Étape 2 : endpoints = les 3 adresses de pods en 8080, et la page d'accueil répond via le port-forward sur le port 80 du Service. Étape 3 : kubectl scale ou réapplication du manifeste → 5 pods Running, créés en quelques secondes. Étape 4 : le rollout montre les deux ReplicaSets se croiser sans que le nombre de pods Ready ne descende sous 5 (maxUnavailable: 0) ; kubectl rollout history montre les deux révisions. Étape 5 : le pod supprimé est recréé par le ReplicaSet en quelques secondes, avec un nouveau nom et une nouvelle adresse IP — preuve concrète de la boucle de réconciliation et de l'éphémère nature des pods. Étape 6 : endpoints <none> malgré des pods Running, puis retour des adresses après correction du sélecteur ; l'explication attendue cite la règle « le sélecteur doit correspondre exactement aux labels des pods ». Le compte rendu est réussi si chaque sortie est expliquée par le mécanisme correspondant (réconciliation, sélecteur, stratégie de mise à jour) et non seulement recopiée.

Exercices corrigés supplémentaires

Exercice 1

Un Deployment de 10 réplicas est configuré avec maxSurge: 25% et maxUnavailable: 25%. Pendant la mise à jour roulante, combien de pods au maximum peuvent exister simultanément, et combien de pods au minimum restent disponibles ? Justifie les arrondis.

✔ Corrigé
maxSurge : 25 % de 10 = 2,5, arrondi au SUPÉRIEUR → 3 pods supplémentaires possibles, soit 13 pods au maximum pendant la transition. maxUnavailable : 25 % de 10 = 2,5, arrondi à l'INFÉRIEUR → 2 pods indisponibles tolérés, soit au minimum 8 pods disponibles. Les arrondis vont toujours dans le sens de la promesse de disponibilité : on autorise un peu plus de marge de manœuvre en création, jamais plus d'indisponibilité que ce qui a été demandé.

Exercice 2

Voici un manifeste fautif : un Secret dont la valeur du mot de passe est écrite telle quelle (`mot-de-passe: admin123`) sous la clé `data:`, et le fichier est sur le point d'être commité dans le dépôt public du projet. Explique les deux erreurs et donne la version corrigée du champ data pour le mot de passe « admin123 » (à titre d'exemple), ainsi que la mesure à prendre pour le dépôt.

✔ Corrigé
Erreur n° 1 : sous `data:`, les valeurs doivent être encodées en base64 ; en clair, le manifeste est rejeté ou mal lu. admin123 en base64 donne YWRtaW4xMjM=, donc la ligne corrigée est `mot-de-passe: YWRtaW4xMjM=` (ou bien on utilise stringData pour écrire en clair dans le manifeste appliqué, Kubernetes encodant alors lui-même). Erreur n° 2, la plus grave : base64 n'est pas un chiffrement — n'importe quel lecteur du dépôt public récupère le mot de passe en une commande. Le fichier ne doit donc jamais être commité : on l'exclut par .gitignore, on crée le Secret directement par kubectl create secret ou via un gestionnaire de secrets, et si le mot de passe a déjà été exposé, on le considère comme compromis et on le change (rotation) avant tout usage en production.

Exercice 3

Deux équipes défendent des choix opposés pour le même projet : une application interne de gestion de congés, 40 utilisateurs, hébergée sur le serveur unique de l'institut. L'équipe A veut un cluster Kubernetes « pour être prêts à grandir », l'équipe B propose Docker Compose. En t'appuyant sur les critères de la leçon 6, départage-les et propose une trajectoire d'évolution.

✔ Corrigé
L'équipe B a raison pour aujourd'hui : 40 utilisateurs, un seul serveur, une seule application — aucun des moteurs de Kubernetes (auto-réparation multi-machines, autoscaling, déploiements de nombreuses équipes) n'est sollicité, tandis que ses coûts (compétences, exploitation du plan de contrôle, astreinte) seraient bien réels pour un institut. Docker Compose sur le serveur répond au besoin complet : définition déclarative des deux ou trois conteneurs, redémarrage automatique, sauvegardes simples. Le point valable de l'équipe A — préparer l'avenir — se satisfait autrement : conteneuriser proprement l'application dès maintenant (image versionnée, configuration externalisée, données dans des volumes) rend une migration future vers Kubernetes ou des conteneurs managés presque mécanique. Trajectoire : Compose aujourd'hui ; si l'application devient critique ou si d'autres services la rejoignent sur le même parc, réévaluer avec des conteneurs managés ou un k3s ; Kubernetes complet seulement quand plusieurs équipes et de vraies exigences de haute disponibilité le justifieront.

QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. Quelle est la plus petite unité que Kubernetes déploie et ordonnance ?

   A) Le conteneur

   B) Le pod

   C) Le nœud

2. Travailler de façon déclarative avec Kubernetes signifie…

A) Donner des ordres pas à pas au cluster dans le bon ordre

B) Décrire l'état désiré et laisser les contrôleurs réconcilier l'état réel en permanence

C) Écrire ses conteneurs directement sur chaque nœud

3. Un Service affiche « Endpoints: <none> » alors que ses pods tournent. La cause la plus probable est…

A) Le Service est de type ClusterIP

B) Le cluster manque de mémoire

C) Le sélecteur du Service ne correspond aux labels d'aucun pod

4. Dans une mise à jour roulante, maxUnavailable: 0 garantit…

A) Qu'aucun nouveau pod n'est créé pendant la mise à jour

B) Qu'à tout moment, le nombre de pods disponibles ne descend pas sous le nombre de réplicas voulu

C) Que la mise à jour est instantanée

5. Un Secret Kubernetes, par défaut, est…

A) Chiffré de bout en bout par Kubernetes

B) Invisible même pour les administrateurs du cluster

C) Simplement encodé en base64, donc lisible par quiconque peut le lire — le chiffrement au repos doit être activé en plus

6. À quoi sert la readiness probe, par opposition à la liveness probe ?

A) À redémarrer un conteneur bloqué

B) À décider si le pod peut recevoir du trafic du Service

C) À mesurer la consommation mémoire du pod

7. Après le déploiement d'une version défaillante, la commande de retour en arrière la plus directe est…

A) kubectl delete deployment puis tout recréer

B) kubectl rollout undo, qui restaure la révision précédente du modèle de pod

C) docker rollback, exécutée sur chaque nœud

8. Pour une application unique sur une seule machine, tenue par une petite équipe, le choix le plus proportionné est en général…

A) Un cluster Kubernetes multi-nœuds

B) Un opérateur Kubernetes développé sur mesure

C) Docker Compose sur la machine, avec des sauvegardes soignées

9. Quelle affirmation sur les tags d'image est correcte ?

A) Le tag « latest » garantit de déployer toujours la même version

B) Un tag peut être déplacé vers une autre image ; le digest, lui, identifie un contenu unique

C) Un digest change à chaque poussée de la même image

10. Quel composant faut-il soupçonner en premier quand un pod reste bloqué en Pending ?

A) Le planificateur (scheduler), qui ne trouve aucun nœud capable d'accueillir le pod avec ses demandes de ressources

B) Le Service, qui bloque le démarrage des pods

C) La ConfigMap, qui retarde l'ordonnancement

Corrigés du QCM

1. Réponse B — Le pod — un ou plusieurs conteneurs partageant réseau et volumes — est l'unité atomique de déploiement ; Kubernetes ne place jamais un conteneur isolé.

2. Réponse B — On dépose des manifestes d'état désiré ; les boucles de réconciliation corrigent tout écart, y compris après une panne.

3. Réponse C — Sans correspondance exacte entre sélecteur et labels, le Service n'a aucune cible : c'est le piège classique, sans aucun message d'erreur.

4. Réponse B — maxUnavailable borne l'indisponibilité : à 0, un ancien pod ne s'éteint que lorsqu'un nouveau est prêt à le remplacer.

5. Réponse C — base64 est un encodage, pas un chiffrement : RBAC strict, chiffrement d'etcd et gestionnaire externe complètent la protection.

6. Réponse B — La readiness probe branche ou retire le pod du trafic ; la liveness probe, elle, décide de redémarrer le conteneur.

7. Réponse B — Le Deployment conserve l'historique de ses révisions : rollout undo réapplique la précédente en quelques secondes.

8. Réponse C — Sans besoin d'auto-réparation multi-machines ni d'autoscaling, Compose répond complètement au besoin pour un coût d'exploitation minimal.

9. Réponse B — Le tag est une étiquette mutable ; le digest sha256 est l'empreinte immuable du contenu — en production on épingle des versions explicites.

10. Réponse A — Pending signifie « pas encore placé » : en général les requests dépassent la capacité libre des nœuds, ou une contrainte de placement n'est satisfaite nulle part — kubectl describe pod le confirme par ses événements.

Résumé — les notions importantes

• Image, conteneur, registre : l'image est le modèle immuable identifié par un digest, le conteneur son instance vivante, le registre leur bibliothèque partagée — Docker lance, mais n'orchestre pas.

• Kubernetes fonctionne en mode déclaratif : on décrit l'état désiré dans des manifestes, et des contrôleurs réconcilient l'état réel en permanence.

• Le pod est la plus petite unité déployable ; le Deployment maintient le nombre voulu de réplicas d'un modèle de pod et gère leurs remplacements.

• Le Service donne un nom stable aux pods et les choisit par un sélecteur de labels : sans correspondance exacte, le Service n'a aucun endpoint et le trafic n'arrive jamais.

• La mise à jour roulante (maxSurge, maxUnavailable, readiness probe) remplace les pods sans coupure, et kubectl rollout undo offre le retour en arrière immédiat.

• ConfigMap pour la configuration, Secret pour les données sensibles — en sachant qu'un Secret n'est que du base64 par défaut : RBAC, chiffrement au repos et rotation sont les vrais garde-fous.

• Superviser, c'est croiser métriques, journaux sur stdout et sondes ; et choisir son outil reste un arbitrage : Compose ou services managés quand Kubernetes serait disproportionné.

Fiches de révision express

Chaque fiche reprend les points « À retenir » d'une leçon. Relis-les la veille d'un contrôle : cinq minutes par fiche suffisent.

Fiche 1 — Rappels Docker : images, conteneurs et registres
Une image est immuable et en lecture seule ; un conteneur ajoute une couche inscriptible temporaire par-dessus. ; Un tag est une étiquette modifiable, un digest est une empreinte unique : en production on déploie des versions explicites, jamais « latest ». ; Docker Compose suffit pour quelques conteneurs sur une machine ; l'orchestration commence quand il faut réparer, répartir et mettre à jour automatiquement.

Fiche 2 — Orchestration avec Kubernetes : pods et services
Le pod est la plus petite unité déployable de Kubernetes ; il est éphémère et son adresse IP change à chaque remplacement. ; Le sélecteur d'un Service (ou d'un Deployment) doit correspondre EXACTEMENT aux labels du modèle de pod, sinon l'objet est vide de cibles. ; État désiré + boucles de réconciliation : après un incident, Kubernetes revient tout seul à la cible déclarée, sans intervention humaine.

Fiche 3 — Déploiements et mises à jour sans interruption
maxSurge = pods supplémentaires autorisés pendant la mise à jour ; maxUnavailable = pods indisponibles tolérés en même temps. ; Sans readiness probe, Kubernetes envoie du trafic à des pods démarrés mais pas encore prêts : erreurs garanties pendant les déploiements. ; Chaque modification du modèle de pod crée une révision : kubectl rollout undo revient à la précédente en quelques secondes.

Fiche 4 — Configuration et secrets dans les environnements conteneurisés
La même image sert partout ; seuls ConfigMap et Secrets changent selon l'environnement. ; base64 n'est pas un chiffrement : kubectl get secret -o yaml révèle les valeurs à quiconque a les droits de lecture — RBAC et chiffrement au repos sont indispensables. ; Un secret qui a fuité ne se « cache » pas, il se remplace : rotation = nouvelles valeurs, redéploiement, révocation des anciennes.

Fiche 5 — Supervision et journalisation d'un cluster
Les journaux vont sur stdout/stderr : tout fichier écrit dans le conteneur disparaît avec lui. ; CrashLoopBackOff = le conteneur démarre puis meurt en boucle ; kubectl logs --previous lit le journal du conteneur mort, describe montre les événements. ; Une liveness probe trop agressive transforme un ralentissement en boucle de redémarrages : elle se règle avec autant de soin que le code.

Fiche 6 — Limites de l'orchestration et alternatives plus légères
Le coût d'un cluster ne se mesure pas en licences mais en compétences et en temps d'exploitation, nuit et week-end compris. ; Docker Compose n'est pas un « sous-Kubernetes » : c'est la bonne réponse, complète, pour une machine et quelques services. ; Migrer de Compose vers Kubernetes reste possible quand le besoin grandit ; l'inverse (un cluster surdimensionné abandonné) est le vrai échec.

Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

Leçon 1 — DOCKER 101. Maîtriser les bases de docker

Chaîne : chillotech · Langue : Français · Niveau : Débutant à intermédiaire
Un cours complet et pratique sur les fondamentaux de Docker : installation, premières commandes, création d'images et déploiement de conteneurs. Parfait pour remettre à niveau tout ce que la leçon 1 résume avant d'attaquer l'orchestration.

https://www.youtube.com/watch?v=jXBS9-lvyvY

Leçon 1 — Docker débutant

Chaîne : PlaisirArduino · Langue : Français · Niveau : Débutant
Une prise en main courte et concrète de Docker pour les grands débutants : images, conteneurs et commandes essentielles démontrées pas à pas. Idéale en toute première approche, avant le cours chillotech plus approfondi.

https://www.youtube.com/watch?v=uqHG3VaGybU

Leçon 2 — Kubernetes de A à Z – Formation Complète 7H

Chaîne : Donald Programmeur · Langue : Français · Niveau : Débutant à intermédiaire
La formation de référence en français pour couvrir tout le cours : architecture du cluster, pods, services, déploiements, ConfigMaps et Secrets, puis monitoring. Travaille-la par chapitres, en parallèle des leçons correspondantes.

https://www.youtube.com/watch?v=yFp_BxFnIO4

Leçon 2 — Kubernetes E03 - Exposer une application, Service et Ingress

Chaîne : Bitdyne · Langue : Français · Niveau : Intermédiaire
Épisode entièrement consacré au problème des adresses changeantes des pods et à sa solution : le Service (ClusterIP, NodePort, LoadBalancer) et l'Ingress. Le complément direct de la leçon 2 et de son piège du sélecteur.

https://www.youtube.com/watch?v=x6sRNcyZatY

Leçon 3 — Understanding Kubernetes in a visual way - 02 - Deployments | Kubernetes en français

Chaîne : Aurelie Vache · Langue : Français · Niveau : Débutant à intermédiaire

Les Deployments expliqués en sketchnotes animées : ReplicaSets, réplicas et mises à jour dessinés plutôt que seulement décrits. Une façon très visuelle de fixer le mécanisme de la leçon 3 avant de passer aux travaux pratiques.

https://www.youtube.com/watch?v=9dyiSiyET3I

Sources et références

• Programme officiel Licence Informatique, Université IAM (Institut Africain de Management), Bamako — vérifier les intitulés d'UE auprès du secrétariat pédagogique.

• Documentation officielle Kubernetes (kubernetes.io, avec sa version française) : concepts (pods, services, déploiements), tâches et tutoriels interactifs — la source primaire pour chaque objet vu dans ce cours.

• Documentation officielle Docker (docs.docker.com) : images et Dockerfile, registres, Docker Compose — le complément naturel de la leçon 1 et des alternatives de la leçon 6.

• Burns, Beda & Hightower — « Kubernetes: Up and Running » (O'Reilly) : l'ouvrage de référence écrit par des créateurs du projet, pour approfondir après ce cours d'initiation.

• Cloud Native Computing Foundation (cncf.io) : l'écosystème des outils cloud native, les projets gradués (Kubernetes, Prometheus, Fluentd) et les parcours de certification (CKA, CKAD).

• Les vidéos listées dans la section « Vidéos associées aux leçons » de ce document (liens vérifiés via YouTube au moment de la publication).

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !

Mes notes

Utilise ces pages pour recopier les définitions, refaire les calculs et noter tes questions pour le prochain cours.